DSA Streak: Daily LeetCode in Java

2026-11-10 | 5 problems

#	Problem	Difficulty	Topic
1	Climbing Stairs II	Medium	Array, Dynamic Programming
2	Valid Parentheses	Easy	String, Stack
3	Search in Rotated Sorted Array	Medium	Array, Binary Search
4	Course Schedule	Medium	Graph, Topological Sort
5	Merge k Sorted Lists	Hard	Linked List, Heap (Priority Queue)

1. Climbing Stairs II

Medium | #3693 | Array, Dynamic Programming | Problem of the Day
https://leetcode.com/problems/climbing-stairs-ii/

Problem

There are steps 0 to n. You start on step 0 with cost 0. From step i you may jump to i+1, i+2 or i+3. Landing on step j from step i costs costs[j] + (j - i)^2, where costs is 1-indexed. Find the minimum total cost to reach step n.

Examples

Example 1	
Input	`n = 4, costs = [1,2,3,4]`
Output	`13`
Why	Path 0 -> 1 -> 2 -> 4 costs (1+1) + (2+1) + (4+4) = 2 + 3 + 8 = 13.

Example 2	
Input	`n = 4, costs = [5,1,6,2]`
Output	`11`
Why	Path 0 -> 2 -> 4 costs (1+4) + (2+4) = 5 + 6 = 11.

Example 3	
Input	`n = 3, costs = [9,8,3]`
Output	`12`
Why	Jump straight 0 -> 3: costs[3] + 3^2 = 3 + 9 = 12.

Intuition

The cheapest way to reach step j must come from one of only three places: j-1, j-2 or j-3. If we already know the cheapest cost to reach each of those, the best for j is the minimum of (best of previous step + costs[j] + jump^2). Each step only looks back three positions, so a simple left to right DP works.

Approach

1. Create dp of size n+1, dp[0] = 0 (we start on step 0 for free).
2. For each step j from 1 to n, set dp[j] to infinity.
3. Try jump = 1, 2, 3 while j - jump >= 0: candidate = dp[j - jump] + costs[j - 1] + jump * jump (costs is 1-indexed in the statement, so step j uses costs[j - 1] in Java).
4. Keep the minimum candidate in dp[j].
5. Return dp[n]. A long array is used so no intermediate sum can overflow.

Dry run on Example 1

- n = 4, costs = [1,2,3,4], dp[0] = 0
- j = 1: jump 1 -> dp[0] + 1 + 1 = 2. dp[1] = 2
- j = 2: jump 1 -> dp[1] + 2 + 1 = 5; jump 2 -> dp[0] + 2 + 4 = 6. dp[2] = 5
- j = 3: jump 1 -> dp[2] + 3 + 1 = 9; jump 2 -> dp[1] + 3 + 4 = 9; jump 3 -> dp[0] + 3 + 9 = 12. dp[3] = 9
- j = 4: jump 1 -> dp[3] + 4 + 1 = 14; jump 2 -> dp[2] + 4 + 4 = 13; jump 3 -> dp[1] + 4 + 9 = 15. dp[4] = 13
- Return dp[4] = 13 (matches the path 0 -> 1 -> 2 -> 4).

Java solution

```
class Solution {
    public int climbStairs(int n, int[] costs) {
        // dp[j] = minimum total cost to stand on step j (step 0 costs nothing)
        long[] dp = new long[n + 1];
```

```
        for (int j = 1; j <= n; j++) {
            dp[j] = Long.MAX_VALUE;
            // the previous step can be j-1, j-2 or j-3 (never below step 0)
            for (int jump = 1; jump <= 3 && j - jump >= 0; jump++) {
                long cost = dp[j - jump] + costs[j - 1] + (long) jump * jump;
                dp[j] = Math.min(dp[j], cost);
            }
        }
        return (int) dp[n];
    }
}
```

Complexity

Time: O(n), 3 transitions per step Space: O(n) for dp (can be cut to O(1) by keeping only the last 3 values)

Edge cases handled

- n = 1: only a single jump of 1 is possible.
- Early steps (j = 1, 2) where some jumps would go below step 0 are skipped by the j - jump >= 0 check.
- Very expensive middle steps: a jump of 3 can skip them even though it costs 9 extra.
- n = 10^5 with costs 10^4: the sum stays near 10^9, computed in long to be safe.

Pattern to remember

"Reach position j from a few fixed previous positions with a cost" means 1D DP: dp[j] = min over the allowed previous states (the Min Cost Climbing Stairs / Frog Jump family).

2. Valid Parentheses

Easy | #20 | String, Stack
https://leetcode.com/problems/valid-parentheses/

Problem

Given a string made of the characters ( ) [ ] { }, decide whether every bracket is closed by the same type of bracket, in the correct order, and every closing bracket has a matching opener.

Examples

Example 1	
Input	`s = "()"`
Output	`true`
Why	One pair, correctly closed.

Example 2	
Input	`s = "()[]{}"`
Output	`true`
Why	Three separate pairs.

Example 3	
Input	`s = "(]"`
Output	`false`
Why	(is closed by the wrong type.

Example 4	
Input	`s = "([])"`
Output	`true`
Why	[] is nested properly inside ().

Example 5	
Input	`s = "([)]"`
Output	`false`
Why	(is closed before [which was opened later, wrong order.

Intuition

The most recently opened bracket must be the first one closed. That is exactly Last In First Out, so a stack fits. A small trick: when we see an opener, push the closer we expect. Then a closer is valid only if it equals the top of the stack.

Approach

1. If the length is odd, return false right away.
2. Scan each character.
3. If it is an opener, push its matching closer onto the stack.
4. If it is a closer, the stack must be non-empty and its popped top must equal this character; otherwise return false.
5. At the end, the string is valid only if the stack is empty (no unclosed openers).

Dry run on Example 1

- s = "()", length 2 is even, stack = []
- ch = '(' : opener, push ')'. stack = [')']
- ch = ')' : closer, stack not empty, pop ')' which equals ch, continue. stack = []
- End of string, stack is empty, return true.

Java solution

```
import java.util.ArrayDeque;
import java.util.Deque;

class Solution {
    public boolean isValid(String s) {
        // odd length can never be fully matched
        if (s.length() % 2 == 1) return false;
        Deque<Character> stack = new ArrayDeque<>();
        for (char ch : s.toCharArray()) {
            // for an opener, push the closer we expect to see later
            if (ch == '(') stack.push(')');
            else if (ch == '[') stack.push(']');
            else if (ch == '{') stack.push('}');
            else if (stack.isEmpty() || stack.pop() != ch) return false;
        }
        return stack.isEmpty();
    }
}
```

Complexity

Time: O(n) Space: O(n) for the stack in the worst case

Edge cases handled

- Single character like "(" or "]": odd length, false.
- Closer with an empty stack such as "))((": false immediately.
- Only openers like "((": loop finishes but stack is not empty, false.
- Deep nesting of length 10^4 works since ArrayDeque grows as needed.

Pattern to remember

Matching pairs where the latest open must close first means a stack. Pushing the expected closer keeps the comparison to a single equality check.

3. Search in Rotated Sorted Array

Medium | #33 | Array, Binary Search
https://leetcode.com/problems/search-in-rotated-sorted-array/

Problem

A sorted array of distinct numbers may have been rotated at some unknown point, for example [0,1,2,4,5,6,7] becomes [4,5,6,7,0,1,2]. Return the index of target, or -1 if it is not there, in O(log n) time.

Examples

Example 1	
Input	`nums = [4,5,6,7,0,1,2], target = 0`
Output	`4`
Why	0 sits at index 4.

Example 2	
Input	`nums = [4,5,6,7,0,1,2], target = 3`
Output	`-1`
Why	3 is not in the array.

Example 3	
Input	`nums = [1], target = 0`
Output	`-1`
Why	Single element that is not the target.

Intuition

When you cut a rotated sorted array at mid, at least one of the two halves is still fully sorted. Comparing nums[left] with nums[mid] tells you which one. For a sorted half you can check in O(1) whether target lies inside its range, so you know which half to throw away. That keeps the normal binary search halving.

Approach

1. left = 0, right = n - 1.
2. While left <= right: mid = left + (right - left) / 2. If nums[mid] == target, return mid.
3. If nums[left] <= nums[mid], the left half is sorted: if nums[left] <= target < nums[mid], search left (right = mid - 1), else search right (left = mid + 1).
4. Otherwise the right half is sorted: if nums[mid] < target <= nums[right], search right (left = mid + 1), else search left (right = mid - 1).
5. If the loop ends, return -1.

Dry run on Example 1

- nums = [4,5,6,7,0,1,2], target = 0, left = 0, right = 6
- mid = 3, nums[3] = 7 != 0. nums[0] = 4 <= 7, left half sorted. Is 4 <= 0 < 7? No, so left = 4.
- left = 4, right = 6, mid = 5, nums[5] = 1 != 0. nums[4] = 0 <= 1, left half sorted. Is 0 <= 0 < 1? Yes, so right = 4.
- left = 4, right = 4, mid = 4, nums[4] = 0 == target. Return 4.

Java solution

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
```

```
            if (nums[mid] == target) return mid;
            if (nums[left] <= nums[mid]) {
                // left half [left..mid] is sorted
                if (nums[left] <= target && target < nums[mid]) right = mid - 1;
                else left = mid + 1;
            } else {
                // right half [mid..right] is sorted
                if (nums[mid] < target && target <= nums[right]) left = mid + 1;
                else right = mid - 1;
            }
        }
        return -1;
    }
}
```

Complexity

Time: O(log n) Space: O(1)

Edge cases handled

- Single element, found or not found.
- Array not rotated at all: behaves like normal binary search.
- Two elements such as [3,1]: the <= in nums[left] <= nums[mid] handles left == mid.
- Negative values and target at the pivot (the first element of the second run).

Pattern to remember

In a rotated sorted array, one half around mid is always sorted. Find that half, check if the target falls in its range, and discard the other half.

4. Course Schedule

Medium | #207 | Graph, Topological Sort, BFS
https://leetcode.com/problems/course-schedule/

Problem

There are numCourses courses labeled 0 to numCourses - 1. A pair [a, b] means you must take b before a. Return true if it is possible to finish all courses, which is the same as asking whether the prerequisite graph has no cycle.

Examples

Example 1	
Input	`numCourses = 2, prerequisites = [[1,0]]`
Output	`true`
Why	Take course 0, then course 1.

Example 2	
Input	`numCourses = 2, prerequisites = [[1,0],[0,1]]`
Output	`false`
Why	Course 1 needs 0 and course 0 needs 1, a cycle, so it is impossible.

Intuition

Treat courses as nodes and draw an edge b -> a for each [a, b]. A course with no remaining prerequisites (indegree 0) can be taken now. Taking it removes its outgoing edges, which may free other courses. If we can take every course this way the graph has no cycle. Courses inside a cycle always keep indegree at least 1, so they are never taken.

Approach

1. Build an adjacency list graph[b] containing a for each [a, b], and count indegree[a].
2. Push every course with indegree 0 into a queue.
3. Pop a course, increment taken, and decrease the indegree of each course it unlocks; when one drops to 0, push it.
4. When the queue is empty, return taken == numCourses.

Dry run on Example 1

- numCourses = 2, prerequisites = [[1,0]]
- graph[0] = [1], graph[1] = [], indegree = [0, 1]
- queue = [0] (only course 0 has indegree 0), taken = 0
- poll 0, taken = 1. Neighbor 1: indegree[1] becomes 0, push 1. queue = [1]
- poll 1, taken = 2. No neighbors. queue empty.
- taken (2) == numCourses (2), return true.

Java solution

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.List;
import java.util.Queue;

class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        // edge b -> a means "finish b before a"
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
```

```
        int[] indegree = new int[numCourses];
        for (int[] pre : prerequisites) {
            graph.get(pre[1]).add(pre[0]);
            indegree[pre[0]]++;
        }
        // Kahn's algorithm: start with courses that need nothing
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < numCourses; i++) if (indegree[i] == 0) queue.offer(i);
        int taken = 0;
        while (!queue.isEmpty()) {
            int course = queue.poll();
            taken++;
            for (int next : graph.get(course)) {
                if (--indegree[next] == 0) queue.offer(next);
            }
        }
        // if a cycle exists, its courses never reach indegree 0
        return taken == numCourses;
    }
}
```

Complexity

Time: O(V + E), V = numCourses, E = prerequisites.length Space: O(V + E) for the graph, indegree array and queue

Edge cases handled

- No prerequisites at all: every course starts with indegree 0, true.
- Self loop [0,0]: course 0 never reaches indegree 0, false.
- Cycle that does not include course 0, such as 1 -> 2 -> 3 -> 1: still detected because taken < numCourses.
- Long chain of 2000 courses: BFS is iterative, so no stack overflow risk.

Pattern to remember

"Can all tasks with dependencies be completed?" means cycle detection in a directed graph. Kahn's algorithm (BFS on indegree 0 nodes) answers it and also gives a valid order.

5. Merge k Sorted Lists

Hard | #23 | Linked List, Heap (Priority Queue), Divide and Conquer
https://leetcode.com/problems/merge-k-sorted-lists/

Problem

You are given k linked lists, each sorted in ascending order. Merge them all into one sorted linked list and return its head.

Examples

Example 1	
Input	`lists = [[1,4,5],[1,3,4],[2,6]]`
Output	`[1,1,2,3,4,4,5,6]`
Why	The lists are 1->4->5, 1->3->4 and 2->6; merging them gives 1->1->2->3->4->4->5->6.

Example 2	
Input	`lists = []`
Output	`[]`
Why	No lists, so the result is empty.

Example 3	
Input	`lists = [[]]`
Output	`[]`
Why	One list that is empty.

Intuition

The next node of the answer is always the smallest among the current heads of the k lists. A min-heap gives that smallest head in O(log k). After taking a node, only its own list's next node becomes a new candidate, so we push just that one. The heap never holds more than k nodes.

Approach

1. Create a min-heap of ListNode ordered by val and push every non-null head.
2. Use a dummy node and a tail pointer to build the answer.
3. While the heap is not empty: poll the smallest node, attach it to tail, move tail forward.
4. If the polled node has a next node, push that next node into the heap.
5. Return dummy.next.

Dry run on Example 1

- Lists A = 1->4->5, B = 1->3->4, C = 2->6. heap = {1(A), 1(B), 2(C)}
- poll 1(A), result 1, push 4(A). heap = {1(B), 2(C), 4(A)}
- poll 1(B), result 1,1, push 3(B). heap = {2(C), 3(B), 4(A)}
- poll 2(C), result 1,1,2, push 6(C). heap = {3(B), 4(A), 6(C)}
- poll 3(B), result 1,1,2,3, push 4(B). heap = {4(A), 4(B), 6(C)}
- poll 4(A), push 5(A); poll 4(B), list B is done. heap = {5(A), 6(C)}
- poll 5(A), then 6(C). Final list 1,1,2,3,4,4,5,6.

Java solution

```
import java.util.PriorityQueue;

/**
 * Definition for singly-linked list.
```

```
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        // min-heap ordered by node value, holds the current head of each list
        PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) -> Integer.compare(a.val, b.v
        al));
        for (ListNode head : lists) {
            if (head != null) heap.offer(head);
        }
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;
        while (!heap.isEmpty()) {
            ListNode smallest = heap.poll();
            tail.next = smallest;
            tail = smallest;
            // the next node from the same list becomes a candidate
            if (smallest.next != null) heap.offer(smallest.next);
        }
        return dummy.next;
    }
}
```

Complexity

Time: O(N log k), N = total nodes, k = number of lists Space: O(k) for the heap (nodes are relinked, not copied)

Edge cases handled

- lists is empty: heap stays empty, return null.
- lists = [[]]: null heads are skipped, return null.
- Some empty lists mixed with non-empty ones.
- Many duplicate values across lists, and negative values.
- k = 10^4 lists of one node each.

Pattern to remember

"Merge k sorted things" or "repeatedly take the smallest of k candidates" means a min-heap of size k. Divide and conquer pairwise merging is the O(1) extra space alternative with the same O(N log k) time.